Parcel Scanner Sim – Performance Testing

Spis treści

1. Test Objective

The objective of the tests is to verify test cases in terms of their impact on application performance. Although the application was designed for small shipping sorting facilities, we want to examine whether the number of elements and network traffic will affect application performance in areas such as hardware resource usage and response time.

2. Environment

Performance tests will be conducted in a test environment on a physical device connected to Android Studio along with the Android Profiler tool in order to analyze hardware resource consumption under different conditions. In performance testing, it is a common practice that if an application performs efficiently on an older device, it will also perform well on newer devices.

An important step to obtain reliable results is to globally disable animations via ADB:

- adb shell settings put global window_animation_scale 0
- adb shell settings put global transition_animation_scale 0
- adb shell settings put global animator_duration_scale 0

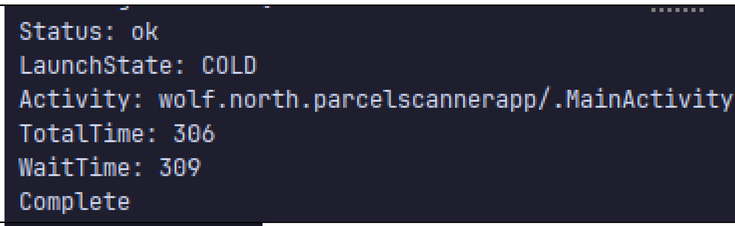
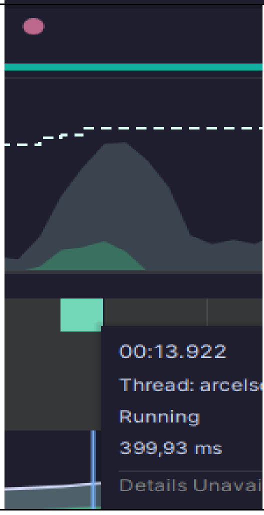
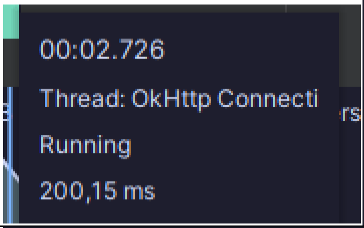
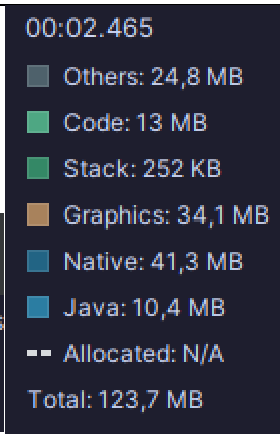
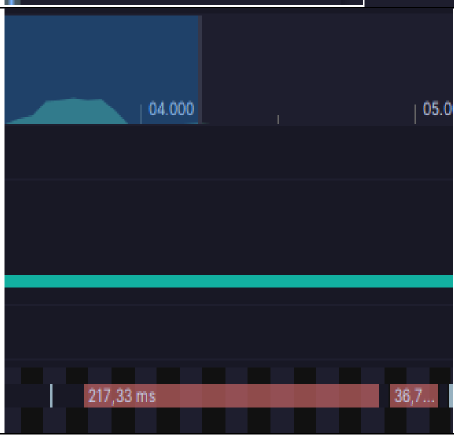
3. Metrics Used in Testing

Considering the size of the application and the number of functionalities, specialized performance automation tools such as UI Automator or Macrobenchmark were not used. Instead, the built-in Android Profiler was used based on correct metrics defined by Google and analysis of the most important test scenarios.

- Cold, Warm, Hot Start
- 16.6ms rule – frame rendering smoothness metric (Jank & Frame Rate)
- Memory metrics and memory management
- CPU load

4. Test Scenarios

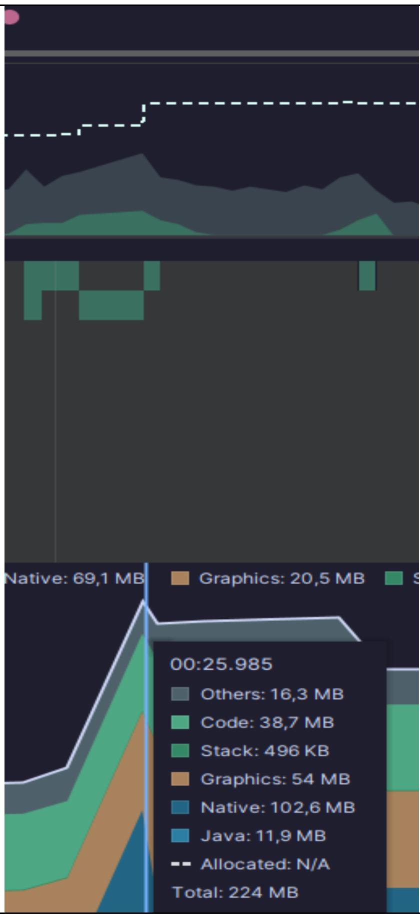
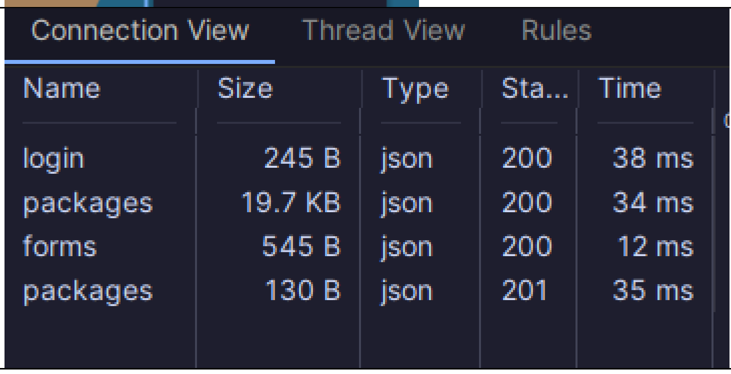
ID Test Case	Name	Steps Description	Metrics	Expected Results
TC-PERF-01	Application launch (Cold Start)	1. Kill the process via ADB 2. Launch the application via ADB 3. Wait for login screen	Startup time	According to Android Vitals, time should be under 5000ms and no white/frozen screens should occur
TC-PERF-02	Successful user login and transition to Home	1. Launch application 2. Log in to the system	API response time & CPU load	Time from login click to Home screen load < 2000ms. No bottlenecks in Main Thread
TC-PERF-03	Successful login with a large assigned list	1. Launch application 2. Log in to the system	API response time (Network) + CPU load (JSON deserialization)	Time from login click to Home screen data display < 2000ms. No Main Thread blocking
TC-PERF-04	Navigation to history screen	1. Launch application 2. Log in 3. Navigate to history screen	UI rendering time	Smooth transition, no stutters. Rendering < 16ms per frame (empty list) or < 100ms (large list – initial render)
TC-PERF-05	Navigation to history with large dataset	1. Launch application 2. Log in 3. Navigate to history screen	UI rendering, janky frames count	Smooth transition, no stuttering despite large dataset
TC-PERF-06	Heavy list scrolling performance test	1. Log in with user having large list 2. Open history screen 3. Scroll rapidly for 10 seconds	Frame rendering (FPS)	16.6ms rule applied, avg 60 FPS. Janky frames < 5%. No frozen frames (>700ms)
TC-PERF-07	Memory usage during history screen load	1. Log in 2. Open history screen 3. Return to Home 4. Force GC in Profiler	Memory allocation	No memory leaks, no continuous growth. GC does not significantly change memory values
TC-PERF-08	Scanner resource usage test	1. Log in 2. Enter package scanning mode 3. Scan barcode 4. Save package	Obciążenie CPU oraz Pamięć	Smooth response < 1000ms. No overheating (Thermal Status OK)
TC-PERF-09	Resource usage during form submission	1. Log in 2. Scan package 3. Save data from bottom sheet	Network Usage	POST request does not block UI, UI remains responsive

Test Case	Description	Result	Interpretation
TC-PERF-01	Application startup time (Cold Start)		Cold start attempts showed results in the range of 250–380ms
TC-PERF-02	Successful login and Home transition		Login and navigation to Home completed in the range of 389–410ms
TC-PERF-03	Login with large dataset		Execution remained within acceptable time ranges, no significant performance degradation observed
TC-PERF-04	Navigation to history screen		Navigation completed within 298–371ms

TC-PERF-05	Navigation with large dataset	Time Range 00:01.789 - 00:02.291 Duration 502,32 ms Data Type Thread ID 21533 States Thread State / Duration / % / Occurrences Running / 301,78 ms / 60,08% / 48 Sleeping / 96,22 ms / 19,16% / 11 No thread activity / 42,95 ms / 8,55% / 1 Runnable / 3,05 ms / 0,61% / 19 Waiting / 1,44 ms / 0,29% / 18 Longest running events (top 10) Start Time / Name / Wall Dura... / Wall Self ... / CPU Dur... / CPU Self ... 00:01.973 / Choreogr... / 211,71 ms / 597 µs / 209,47 ms / 81 µs 00:02.009 / traversal / 175,98 ms / 1,02 ms / 175,3 ms / 342 µs 00:02.009 / measure / 125,99 ms / 812 µs / 125,99 ms / 812 µs 00:02.010 / Android... / 124,95 ms / 25,92 ms / 124,95 ms / 25,92 ms 00:02.192 / Choreogr... / 38,32 ms / 56 µs / 37,74 ms / 80 µs 00:01.973 / animation / 35,07 ms / 79 µs / 34,03 ms / 370 µs 00:01.973 / Recomp... / 34,99 ms / 4,36 ms / 33,66 ms / 3,03 ms 00:02.155 / draw / 30,12 ms / 139 µs / 30,12 ms / 139 µs 00:02.155 / Record V... / 29,98 ms / 29,88 ms / 29,98 ms / 29,88 ms 00:02.192 / animation / 26,7 ms / 154 µs / 26,7 ms / 154 µs	Navigation completed within 298–371ms

TC-PERF-06	Heavy list scrolling performance	`** Graphics info for pid 16411 [wolf.north.parcelscannerapp]`
		
		`Stats since: 693968988120ns`
		`Total frames rendered: 506`
		`Janky frames: 98 (19.37%)`
			Scrolling 176 items resulted in Janky Frames between 18–19.5%

TC-PERF-07	Memory usage test	Package (wolf.north.parcelscannerapp.mvvm.model.files) Courier (wolf.north.parcelscannerapp.mvvm.model.files) Courier[] (wolf.north.parcelscannerapp.mvvm.model.files) ShipmentType[] (wolf.north.parcelscannerapp.mvvm.model.files) ShipmentType (wolf.north.parcelscannerapp.mvvm.model.files) WeightClass[] (wolf.north.parcelscannerapp.mvvm.model.files) WeightClass (wolf.north.parcelscannerapp.mvvm.model.files) Form (wolf.north.parcelscannerapp.mvvm.model.files) Courier$Companion (wolf.north.parcelscannerapp.mvvm.model.files)	History objects were collected by garbage collector after leaving screen, no memory leaks detected

TC-PERF-08	Scanner resource usage		No significant increase in CPU or memory usage; increased threads due to CameraX, parsing, and Retrofit operations
TC-PERF-09	Form submission resource usage		POST request execution and response handling stayed within acceptable performance limits

5. Results Interpretation and Defects

The performance testing process was successful, and no defects requiring immediate fixing were identified. Only in test case TC-PERF-06 can an increased number of Janky Frames be observed (18–19.5%), which should be reported with low priority; however, for an MVP this is considered acceptable.